Reviewer Pack — Minimal Order of Integer-Valued Quasi-Crystals and Circuit M…

Entry 881a213ae900 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 08:15:27 UTC

Minimal Order of Integer-Valued Quasi-Crystals and Circuit Monotone Complexity

Entry ID: `881a213ae900` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-01 08:15:27 UTC

1. Conjecture

Field A (mathematical branch): Quasi-Crystal Theory **Field B** (complexity object): Boolean Circuits

Statement:

> For any given Boolean circuit C with n inputs, the minimal order of an associated integer-valued quasi-crystal (O_qc(C)) is linearly correlated with its monotone complexity (mc(C)), such that O_qc(C) = Θ(mc(C)).

Rationale (proposer's reasoning):

> Quasi-crystals are discrete structures that exhibit long-range order similar to crystals but without periodicity. The integer-valued property of quasi-crystals allows for a direct mapping from circuit structure to the properties of the quasi-crystal. This conjecture suggests that the inherent complexity in the structure of a Boolean circuit can be revealed through the study of its associated quasi-crystal.

Taxonomy category: `QUASICRYSTAL_MONOTONECOMPLEXITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `df99bceb7354ace7`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if at least 80% of the circuits with n ≤ 40 inputs show a correlation coefficient between O_qc(C) and mc(C) greater than or equal to 0.8, and the mean difference in order between O_qc(C) and mc(C) across all circuits is less than or equal to 3.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `'quasi-crystal theory' AND 'boolean circuits' AND 'monotone complexity'` - `'integer-valued quasi-crystals' IN TITLE AND 'Boolean circuit' IN TITLE` - `'circuit monotone complexity' AND 'order of integer-valued quasi-crystals'`

Top relevant hits considered: - [http://arxiv.org/abs/1407.6169v3] Multiplicative Complexity of Vector Valued Boolean Functions - [http://arxiv.org/abs/2202.09350v4] Complexity of warped conformal field theory - [http://arxiv.org/abs/0811.0699v1] A Note on the Inversion Complexity of Boolean Functions in Boolean Formulas

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.3s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction
from itertools import product

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def dpll(circuit):
        if not circuit:
            return True
        for literal in circuit[0]:
            if literal > 0 and literal in assignment:
                continue
            assignment.add(literal)
            if dpll(circuit[1:]):
                return True
            assignment.remove(literal)
            if -literal in assignment:
                continue
            assignment.add(-literal)
            if dpll(circuit[1:]):
                return True
            assignment.remove(-literal)
        return False

    def monotone_complexity(circuit):
        assignment = set()
        return sum(1 for clause in circuit if any(lit > 0 and lit in assignment or -lit not in ass
```

```python
def quasi_crystal_order(circuit):
    n = len(circuit)
    if n == 0:
        return 0
    lattice_points = set()
    for i, clause in enumerate(circuit):
        for literal in clause:
            lattice_points.add((i, literal))
    order = 0
    for point in lattice_points:
        neighbors = [(point[0] + dx, point[1]) for dx in [-1, 1]]
        if all(neighbor in lattice_points for neighbor in neighbors):
            order += 1
    return order

def generate_circuit(n):
    circuit = []
    for _ in range(random.randint(2, n)):
        clause = [random.choice([-1, 1]) * (i + 1) for i in random.sample(range(n), random.ran
        circuit.append(clause)
    return circuit

instances_tested = 0
total_order = 0
total_complexity = 0
n_max = 5

for n in [5, 10, 15, 20, 30, 40]:
    if n > n_max:
        n_max = n

    for _ in range(5):
        circuit = generate_circuit(n)
        instances_tested += 1
        complexity = monotone_complexity(circuit)
        order = quasi_crystal_order(circuit)
        total_order += order
        total_complexity += complexity

if instances_tested == 0:
    return {
        "metric_name": "quasi_crystal_order",
        "metric_value": 0,
        "instances_tested": 0,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "empty_circuit"
    }

mean_order = total_order / instances_tested
mean_complexity = total_complexity / instances_tested

if instances_tested < 30:
    return {
        "metric_name": "quasi_crystal_order",
```

```
            "metric_value": 0,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "insufficient_instances"
        }

    correlation_coefficient = (mean_order * mean_complexity - total_order * total_complexity / ins
                               math.sqrt((total_order**2 / instances_tested - mean_order**2) *
                                         (total_complexity**2 / instances_tested - mean_complexit

    return {
        "metric_name": "quasi_crystal_order",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": abs(correlation_coefficient) >= 0.8 and abs(mean_order - mean_complexi
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        for r in results:
            if not r["conjecture_holds"]:
                counterexample = f"n={r['instances_tested']}, order={total_order}, complexity={tot
                print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={
                break
        else:
            print("RESULT: INCONCLUSIVE insufficient_data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
holds': False, 'counterexample': ''}
TRIAL:  {'metric_name':  'quasi_crystal_order',  'metric_value':  6.490265564577939e-
18, 'instances_tested': 30, 'n_max': 40, 'conjecture_holds': False, 'counterexample': ''}
```

```
TRIAL: {'metric_name': 'quasi_crystal_order', 'metric_value': 0.0, 'instances_tested': 30, 'n_max': 4
TRIAL: {'metric_name': 'quasi_crystal_order', 'metric_value': 0.0, 'instances_tested': 30, 'n_max': 4
TRIAL: {'metric_name': 'quasi_crystal_order', 'metric_value': 0.0, 'instances_tested': 30, 'n_max': 4
TRIAL: {'metric_name': 'quasi_crystal_order', 'metric_value': 0.0, 'instances_tested': 30, 'n_max': 4
TRIAL: {'metric_name': 'quasi_crystal_order', 'metric_value': 0.0, 'instances_tested': 30, 'n_max': 4
TRIAL:  {'metric_name':  'quasi_crystal_order',  'metric_value':  -3.853644801336899e-
18, 'instances_tested': 30, 'n_max': 40, 'conjecture_holds': False, 'counterexample': ''}
TRIAL:  {'metric_name':  'quasi_crystal_order',  'metric_value':  4.295322428261718e-
18, 'instances_tested': 30, 'n_max': 40, 'conjecture_holds': False, 'counterexample': ''}
TRIAL:  {'metric_name':  'quasi_crystal_order',  'metric_value':  5.444771921533335e-
18, 'instances_tested': 30, 'n_max': 40, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'quasi_crystal_order', 'metric_value': 0.0, 'instances_tested': 30, 'n_max': 4

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_af3db7d4.py", line 138, in <module>
    counterexample = f"n={r['instances_tested']}, order={total_order}, complexity={total_complexity}
                                                         ^^^^^^^^^^^^
NameError: name 'total_order' is not defined
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data that could confirm or falsify the conjecture. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	22133
2	preregistration	ollama_remote	glm4:latest	0	0	9375
3	novelty	ollama_remote	glm4:latest	0	0	8656
4	novelty	ollama_remote	glm4:latest	0	0	16629
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15410
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13752
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12512
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14736
9	judge	ollama_remote	glm4:latest	0	0	11830

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 125033 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/881a213ae900.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/881a213ae900.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/881a213ae900.tar.gz` (if generated)